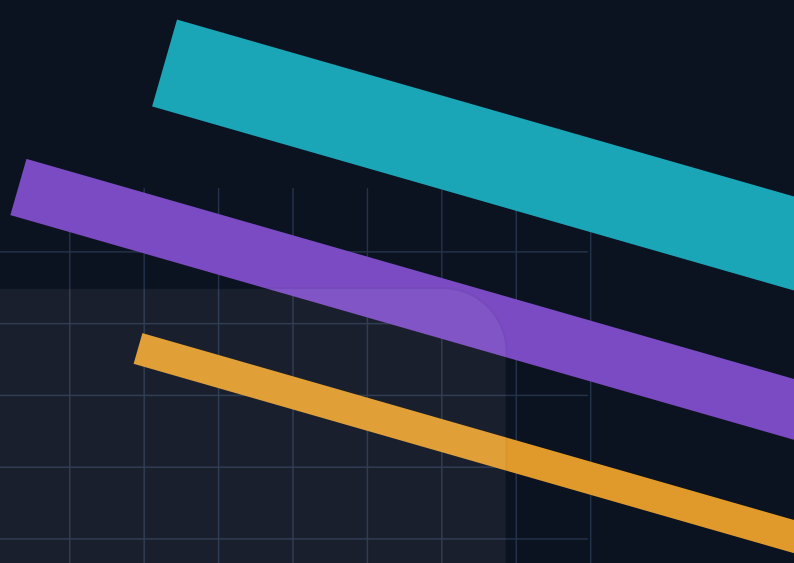




FVGridMaker

Manual da Biblioteca

Biblioteca em C++20 para a geração de malhas estruturadas (1D ou 2D) para o uso em programas de Volumes Finitos.

layout preview 01

uma coluna, componentes laterais



Sumário

Apresentação	3
Malha 1D	5

Apresentação

Ainda hoje, a formulação do Método dos Volumes Finitos mais difundida é a encontrada em Patankar [1]. Embora apresente uma limitação importante, associada ao uso de malhas estruturadas, tais códigos continuam sendo uma das principais portas de entrada para quem começa a desenvolver programas baseados nessa metodologia.

Esta biblioteca computacional foi concebida para quem desenvolve programas baseados no Método dos Volumes Finitos e trabalha com malhas estruturadas. Ela não se propõe a resolver o problema geral de geração de malhas para geometrias complexas; esse não é, nem de longe, o seu propósito. Seu escopo é deliberadamente mais restrito: gerar malhas estruturadas simples, em uma e duas dimensões, para uso em problemas nos quais a regularidade da malha é parte da formulação numérica.

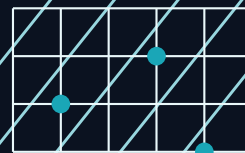
Mesmo com esse escopo limitado, a biblioteca foi elaborada com recursos de C++ moderno para tornar mais simples tanto a geração da malha quanto, principalmente, o acesso às suas informações geométricas. A intenção é que coordenadas de faces, centros, volumes, áreas, índices e relações de vizinhança possam ser consultados de forma direta durante a implementação do modelo numérico da equação diferencial, sem que o código do usuário precise repetir detalhes geométricos de baixo nível.

Esperamos que a FVGridMaker seja útil nas suas tarefas de geração de malhas estruturadas para problemas em uma e duas dimensões. Caso encontre erro, comportamento inesperado ou alguma dificuldade técnica no uso da biblioteca, solicitamos que envie uma mensagem para [livromvf@gmail.com] (mailto:livromvf@gmail.com), preferencialmente acompanhada de uma descrição do problema, do exemplo utilizado e da versão da biblioteca. Faremos o possível para analisar a situação e indicar uma solução adequada.

01

Malha 1D

faces, centros, volumes e inspeção mínima



abertura de capítulo

Resumo editorial

- Construir malhas unidimensionais e acessar suas informações geométricas fundamentais.
- Identificar faces, centros de volume e comprimentos de volume numa malha volume-centrada.
- Comparar malhas uniformes, suavemente concentradas, aleatórias e definidas pelo usuário.
- Avaliar propriedades geométricas simples antes de usar a malha em um método numérico.
- Exportar dados geométricos básicos para inspeção externa.

decisão de leitura

O capítulo começa pelo caso uniforme, que fixa a nomenclatura. Depois apresenta malhas não uniformes, coordenadas fornecidas pelo usuário, reconstruções customizadas e rotinas simples de inspeção. Cada seção acompanha um programa do diretório `manual/` da FVGridMaker.

Como compilar os exemplos deste capítulo

Os comandos abaixo devem ser executados a partir da raiz do repositório da FVGridMaker, isto é, no diretório onde está o arquivo `CMakeLists.txt` principal.

Caso o diretório `build` ainda não exista, configure o projeto com:

A terminal window with a dark background and three colored window control buttons (green, yellow, red) in the top-left corner. The title bar on the right says "configuração". The command `cmake -S . -B build` is entered in the terminal.

```
cmake -S . -B build
```

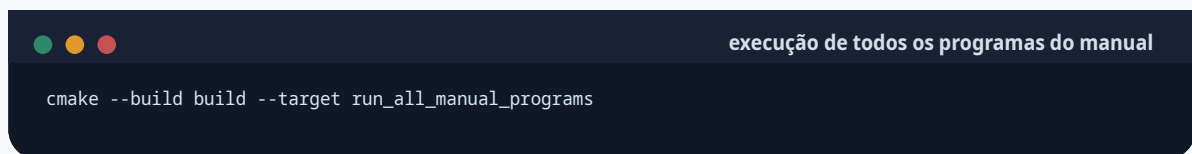
Depois, compile a biblioteca e os exemplos disponíveis com:

A terminal window with a dark background and three colored window control buttons (green, yellow, red) in the top-left corner. The title bar on the right says "compilação". The command `cmake --build build` is entered in the terminal.

```
cmake --build build
```

Os programas do manual usam o prefixo `man_`. Assim, um arquivo chamado `man_uniformGrid.cc` gera o alvo de execução `run_man_uniformGrid`. Ao longo do capítulo, cada seção informa o alvo específico do exemplo.

Também é possível executar todos os programas do manual de uma vez:

A terminal window with a dark background and three colored window control buttons (green, yellow, red) in the top-left corner. The title bar on the right says "execução de todos os programas do manual". The command `cmake --build build --target run_all_manual_programs` is entered in the terminal.

```
cmake --build build --target run_all_manual_programs
```

observação

Diretório de execução

Não é necessário entrar manualmente no diretório `build` para executar os comandos acima. A opção `-B build` indica ao CMake onde ficam os arquivos de compilação, e `cmake --build build` usa esse diretório diretamente.

seção 1.1

Construindo uma malha uniforme

O primeiro exemplo constrói uma malha cartesiana uniforme em uma dimensão. O domínio é o intervalo $[0, 1]$, dividido em 10 volumes finitos de mesmo comprimento. O objetivo é simples: gerar a malha, imprimir suas informações geométricas básicas e verificar se os comprimentos dos volumes fecham o comprimento total do domínio.

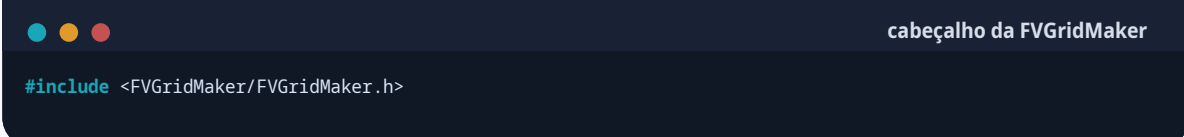
A malha uniforme é o ponto de partida natural porque quase tudo nela é previsível. Se o domínio tem comprimento 1 e foi dividido em 10 volumes, cada volume deve ter comprimento 0,1. Isso facilita conferir se o programa está chamando corretamente a biblioteca.

Leitura do programa

Vamos destacar as partes principais do arquivo `man_uniformGrid.cc`. O código completo pode ser acessado pelo QR code ao final da seção.

Inclusão dos cabeçalhos

Para usar a `FVGridMaker`, o programa inclui o cabeçalho principal da biblioteca:



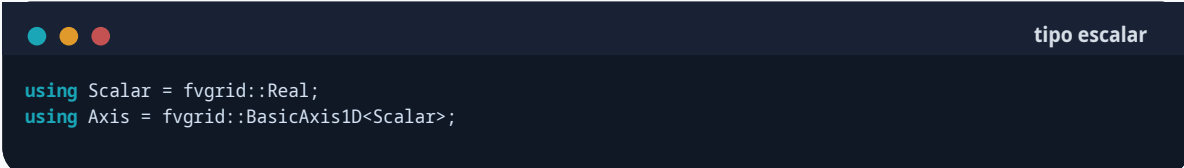
```
1 #include <FVGridMaker/FVGridMaker.h>
```

cabeçalho da FVGridMaker

Esse cabeçalho reúne a interface pública principal da biblioteca. No arquivo completo também aparecem cabeçalhos da biblioteca padrão de C++, como `<iostream>`, `<iomanip>`, `<cmath>` e `<string>`. Eles são usados apenas para impressão, formatação e verificação numérica.

Escolha do tipo escalar

O exemplo usa o tipo real padrão da biblioteca:



```
1 using Scalar = fvgrid::Real;  
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

tipo escalar

O tipo `fvgrid::Real` representa o tipo real padrão da `FVGridMaker`. O apelido `Axis` representa um eixo unidimensional baseado nesse tipo escalar.

Essa forma deixa visível que `BasicAxis1D` é uma classe parametrizada pelo tipo numérico. Se for necessário usar outra precisão, é possível trocar o tipo escalar:

```
1 using Scalar = long double;
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

O restante do programa permanece essencialmente igual.

Definição do domínio

A malha uniforme é definida por três dados:

```
1 const fvgrid::Size nvol = 10;
2 const Scalar xinit = Scalar{0.0};
3 const Scalar xfinal = Scalar{1.0};
```

Aqui, o domínio é $[0, 1]$, dividido em 10 volumes. Como a malha é uniforme, esses três dados determinam todas as faces, todos os centros e todos os comprimentos.

Geração da malha

A malha é criada pela função `uniform_axis_1d`:

```
1 const Axis axis = fvgrid::uniform_axis_1d<Scalar>(
2     nvol,
3     xinit,
4     xfinal
5 );
```

O objeto `axis` passa a armazenar as faces, os centros e as distâncias geométricas associadas à malha. A partir desse ponto, o usuário não precisa reconstruir a geometria manualmente.

Impressão automática

A primeira inspeção pode ser feita com o operador «:

```
1 std::cout << std::fixed << std::setprecision(6);
2 std::cout << axis << '\n';
```

A primeira linha fixa a impressão com seis casas decimais. A segunda imprime a representação textual padrão do objeto `Axis1D`. Essa saída é útil para uma conferência rápida.

Resumo manual da malha

O programa também imprime propriedades globais da malha:

```

1 std::cout << "número de volumes : " << axis.num_cells() << '\n';
2 std::cout << "número de faces   : " << axis.num_faces() << '\n';
3 std::cout << "xmin           : " << axis.xmin() << '\n';
4 std::cout << "xmax           : " << axis.xmax() << '\n';
5 std::cout << "comprimento      : " << axis.length() << '\n';

```

A função `axis.num_cells()` retorna o número de volumes. A função `axis.num_faces()` retorna o número de faces. As funções `axis.xmin()`, `axis.xmax()` e `axis.length()` retornam os limites e o comprimento total do eixo.

Coordenadas das faces

As faces são acessadas por índice:

```

1 for (fvgrid::Size i = 0; i < axis.num_faces(); ++i) {
2     std::cout << std::right
3         << std::setw(id_width) << i
4         << std::setw(value_width) << axis.face(i)
5         << '\n';
6 }

```

Como a malha tem 10 volumes, ela possui 11 faces. A chamada `axis.face(i)` retorna a coordenada da face de índice i .

Informações dos volumes

As informações de cada volume são acessadas pelo índice p :

```

1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2     std::cout << std::right
3         << std::setw(id_width) << p
4         << std::setw(value_width) << axis.west_face(p)
5         << std::setw(value_width) << axis.center(p)
6         << std::setw(value_width) << axis.east_face(p)
7         << std::setw(value_width) << axis.cell_length(p)
8         << '\n';
9 }

```

A chamada `axis.west_face(p)` retorna a face oeste do volume p , `axis.east_face(p)` retorna a face leste, `axis.center(p)` retorna o centro e `axis.cell_length(p)` retorna o comprimento do volume.

Teste de consistência geométrica

O programa fecha com uma verificação simples: a soma dos comprimentos dos volumes deve reproduzir o comprimento total do domínio.

$$\sum_{p=0}^{N-1} \Delta x_p = x_{\max} - x_{\min}.$$

No código, isso aparece como:

```

1 Scalar sum_dx = Scalar{0.0};
2
3 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
4     sum_dx += axis.cell_length(p);
5 }
6
7 const Scalar expected_length = axis.length();
8 const Scalar error = std::abs(sum_dx - expected_length);
9 const Scalar tolerance = Scalar{1.0e-12};

```

teste de consistência

Como os cálculos usam ponto flutuante, o programa compara o erro com uma tolerância pequena, em vez de exigir igualdade exata.

observação

Por que testar uma malha tão simples?

A malha uniforme é simples, mas o teste de consistência estabelece um padrão para os exemplos seguintes. A mesma ideia será usada em malhas não uniformes, onde os comprimentos dos volumes deixam de ser todos iguais.

Como executar este exemplo

```

cmake --build build --target run_man_uniformGrid

```

execução da malha uniforme

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo fixa a nomenclatura básica usada no restante do capítulo. As faces delimitam os volumes, os centros indicam as posições associadas aos volumes, e o comprimento Δx_p mede a extensão de cada volume no eixo x .

seção 1.3

Construindo uma malha aleatória

A malha aleatória é um teste de estresse. O domínio continua sendo $[0, 1]$, dividido em 10 volumes, mas agora as faces internas são sorteadas dentro do intervalo. As faces externas permanecem fixas em 0 e 1.

Essa malha não deve ser vista como uma escolha natural para qualquer problema físico. Aqui ela serve para outra coisa: quebrar a regularidade da malha uniforme e verificar se o programa realmente usa a geometria fornecida pela FVGridMaker.

Leitura do programa

O arquivo correspondente é `man_randomGrid.cc`. Ele segue a mesma estrutura dos exemplos anteriores, mas inclui uma semente para tornar a geração reproduzível.

Cabeçalho e tipos

O programa usa o cabeçalho principal da FVGridMaker:

```
1 #include <FVGridMaker/FVGridMaker.h>
2
3 using Scalar = fvgrid::Real;
4 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

Depois que o objeto `Axis1D` é criado, a forma de consultar a geometria é a mesma usada na malha uniforme e na malha de Roberts.

Domínio e semente

A malha aleatória precisa de uma semente:

```
1 const fvgrid::Size nvol = 10;
2 const Scalar xinit = Scalar{0.0};
3 const Scalar xfinal = Scalar{1.0};
4 const fvgrid::UInt64 seed = 20260627;
```

A semente fixa faz o programa gerar a mesma malha em execuções diferentes. Isso é importante em um manual, porque permite comparar a saída obtida pelo leitor com a saída esperada.

No código completo há também um bloco comentado mostrando como usar uma semente baseada no relógio do sistema. Essa opção é útil para testes exploratórios, mas não é a melhor escolha para documentação reproduzível.

Geração da malha

A malha é criada pela função `random_axis_1d`:

```
1  const Axis axis = fvgrid::random_axis_1d<Scalar>(
2      nvol,
3      xinit,
4      xfinal,
5      seed
6  );
```

As faces externas ficam fixas em `xinit` e `xfinal`. As faces internas são sorteadas no interior do domínio e organizadas em ordem crescente. Depois disso, a biblioteca calcula centros e comprimentos.

Resumo da malha

O programa imprime as propriedades globais e a semente usada:

```
1  std::cout << "número de volumes : " << axis.num_cells() << '\n';
2  std::cout << "número de faces   : " << axis.num_faces() << '\n';
3  std::cout << "xmin           : " << axis.xmin() << '\n';
4  std::cout << "xmax           : " << axis.xmax() << '\n';
5  std::cout << "comprimento    : " << axis.length() << '\n';
6  std::cout << "semente       : " << seed << '\n';
```

A semente é parte do caso de teste. Se ela mudar, a malha sorteada também pode mudar.

Faces e volumes

As faces continuam sendo acessadas por índice:

```
1  for (fvgrid::Size i = 0; i < axis.num_faces(); ++i) {
2      std::cout << std::right
3          << std::setw(id_width) << i
4          << std::setw(value_width) << axis.face(i)
5          << '\n';
6  }
```

E os volumes continuam sendo acessados por p :

```

1  for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2      std::cout << std::right
3          << std::setw(id_width) << p
4          << std::setw(value_width) << axis.west_face(p)
5          << std::setw(value_width) << axis.center(p)
6          << std::setw(value_width) << axis.east_face(p)
7          << std::setw(value_width) << axis.cell_length(p)
8          << '\n';
9  }

```

A diferença aparece nos valores de `axis.cell_length(p)`. Na malha uniforme, todos eram iguais. Na malha aleatória, eles variam de volume para volume.

Teste de consistência geométrica

A soma dos comprimentos dos volumes deve fechar o domínio:

$$\sum_{p=0}^{N-1} \Delta x_p = x_{\max} - x_{\min}.$$

O código acumula os comprimentos e compara o resultado com o comprimento total do eixo:

```

1  Scalar sum_dx = Scalar{0.0};
2
3  for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
4      sum_dx += axis.cell_length(p);
5  }
6
7  const Scalar expected_length = axis.length();
8  const Scalar error = std::abs(sum_dx - expected_length);
9  const Scalar tolerance = Scalar{1.0e-12};

```

Esse teste não prova que a malha é boa para um problema numérico específico. Ele verifica apenas se os volumes cobrem o domínio sem lacunas e sem ultrapassar as extremidades.

observação

Malha aleatória como teste

Se um programa funciona na malha uniforme, mas falha na malha aleatória, é provável que exista alguma hipótese escondida no código. Um erro comum é assumir que Δx é constante sem declarar isso.

Como executar este exemplo

```

1  cmake --build build --target run_man_randomGrid

```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



A malha aleatória fecha o primeiro grupo de exemplos: uniforme, Roberts e aleatória. Em todos eles, a forma de gerar a malha muda, mas a interface de consulta do objeto `Axis1D` permanece a mesma.

seção 1.4

Usando coordenadas fornecidas pelo usuário

Nem sempre a malha será gerada por uma distribuição pronta da biblioteca. Em alguns casos, o usuário já tem as coordenadas em um arquivo, em uma tabela, em uma rotina externa ou em uma construção própria. O exemplo `man_customCoordinates1D.cc` mostra como transformar essas coordenadas em um objeto `Axis1D`.

A ideia é separar dois papéis. Primeiro, o usuário fornece as coordenadas primárias. Depois, um padrão de malha informa como completar a geometria que falta.

Leitura do programa

O exemplo apresenta dois casos: coordenadas de faces fornecidas pelo usuário e coordenadas de centros fornecidas pelo usuário.

Tipos usados no exemplo

O código usa os mesmos apelidos dos exemplos anteriores:

```
1 using Scalar = fvgrid::Real;  
2 using Axis = fvgrid::BasicAxis1D<Scalar>;
```

A diferença não está no tipo do eixo, mas na maneira como ele é construído.

Caso 1: fornecendo as faces

No primeiro caso, o usuário fornece diretamente as faces:

```
1 std::vector<Scalar> user_faces{  
2     0.00, 0.04, 0.10, 0.19, 0.31, 0.46,  
3     0.64, 0.79, 0.90, 0.97, 1.00  
4 };
```

Como há 11 faces, a malha terá 10 volumes. As coordenadas devem estar em ordem crescente e devem representar uma malha válida.

Construção a partir das faces

As faces são empacotadas em um objeto de coordenadas e passadas para `custom_axis_1d`:

```
malha customizada por faces

1 const Axis axis_from_faces = fvgrid::custom_axis_1d(
2   fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(user_faces)),
3   fvgrid::VolumeCentered1D{}
4 );
```

O padrão `fvgrid::VolumeCentered1D{}` informa que as faces são as coordenadas primárias. A biblioteca então calcula os centros e os comprimentos dos volumes.

Esse é o caso mais próximo da formulação usual de volumes finitos: as faces delimitam os volumes, e os centros ficam associados aos volumes.

Caso 2: fornecendo os centros

Também é possível fornecer os centros:

```
centros fornecidos pelo usuário

1 std::vector<Scalar> user_centers{
2   0.03, 0.09, 0.17, 0.28, 0.42,
3   0.58, 0.72, 0.83, 0.91, 0.97
4 };
```

Neste caso, os limites do domínio precisam ser informados separadamente. Sem esses limites, a biblioteca não tem como reconstruir as faces externas.

Domínio limitado

O domínio é criado com `from_bounds`:

```
domínio para centros

1 const auto domain = fvgrid::BasicDomain1D<Scalar>::from_bounds(
2   fvgrid::BasicXInit<Scalar>{xinit},
3   fvgrid::BasicXFinal<Scalar>{xfinal}
4 );
```

Esse domínio fornece as faces externas da malha. Depois disso, o padrão `FaceCentered1D` consegue reconstruir as faces internas a partir dos centros.

Construção a partir dos centros

A construção fica assim:

```
malha customizada por centros

1 const Axis axis_from_centers = fvgrid::custom_axis_1d(
2   fvgrid::BasicCoordinates1D<Scalar>::centers(std::move(user_centers)),
3   fvgrid::FaceCentered1D{},
4   domain
5 );
```

Agora os centros são as coordenadas primárias, e as faces são reconstruídas. O objeto final continua sendo um `Axis1D`, com acesso a faces, centros e comprimentos pelas mesmas funções usadas nos exemplos anteriores.

observação**Coordenadas primárias**

A palavra “primária” não significa mais importante fisicamente. Ela apenas indica qual conjunto de coordenadas foi fornecido primeiro. O padrão de malha decide como calcular o restante da geometria.

Como executar este exemplo

```
execução da malha customizada  
cmake --build build --target run_man_customCoordinates1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo mostra a parte mais flexível da construção 1D: o usuário pode fornecer coordenadas próprias, e a FVGridMaker organiza a geometria em um objeto `Axis1D` consistente.

seção 1.5

Gerando uma malha por lei de potência

O exemplo `man_powerLawGrid.cc` mostra uma maneira simples de criar uma malha por função de mapeamento. Em vez de escrever todas as faces à mão, o programa define uma função que transforma a coordenada computacional η , no intervalo $[0, 1]$, em uma coordenada física x .

Neste caso, a transformação é uma lei de potência. Para $\gamma > 1$, os volumes ficam menores perto de $x = 0$ e maiores à medida que se aproximam de $x = 1$.

Leitura do programa

A ideia central é gerar primeiro as faces com uma função e depois usar `custom_axis_1d` para montar o eixo.

Parâmetros do exemplo

O programa define o domínio, o número de volumes e o expoente da lei de potência:

```
1 const fvgrid::Size nvol = 10;
2 const Scalar xinit = Scalar{0.0};
3 const Scalar xfinal = Scalar{1.0};
4 const Scalar gamma = Scalar{2.0};
5 const Scalar length = xfinal - xinit;
```

parâmetros

Com $\gamma = 2$, a distribuição não é uniforme. As primeiras faces ficam mais próximas entre si, e as últimas ficam mais afastadas.

Função de mapeamento

A função de mapeamento é escrita como uma expressão lambda:

```
1 const auto power_law_map = [xinit, length, gamma](Scalar eta) {
2     return xinit + length * std::pow(eta, gamma);
3 };
```

função de mapeamento

A variável η representa uma coordenada normalizada entre 0 e 1. A função devolve a coordenada física correspondente.

Geração das faces

As faces são geradas a partir do padrão `VolumeCentered1D`:

```

1 std::vector<Scalar> faces =
2   fvgrid::VolumeCentered1D::primary_coordinates_from_map<Scalar>(
3     nvol,
4     power_law_map
5   );

```

Como o padrão `VolumeCentered1D` usa faces como coordenadas primárias, essa chamada gera $N + 1$ faces para N volumes.

Construção do eixo

Depois de gerar as faces, o programa constrói o eixo customizado:

```

1 const Axis axis = fvgrid::custom_axis_1d(
2   fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(faces)),
3   fvgrid::VolumeCentered1D{}
4 );

```

A partir desse ponto, o objeto `axis` é usado como nos exemplos anteriores. A origem das faces mudou, mas a consulta da geometria não mudou.

O que observar na saída

Na tabela dos volumes, os valores de `axis.cell_length(p)` aumentam ao longo do domínio. Isso é esperado, pois a função $x = \eta^\gamma$, com $\gamma = 2$, concentra mais faces perto da origem.

```

1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2   std::cout << std::right
3     << std::setw(id_width) << p
4     << std::setw(value_width) << axis.west_face(p)
5     << std::setw(value_width) << axis.center(p)
6     << std::setw(value_width) << axis.east_face(p)
7     << std::setw(value_width) << axis.cell_length(p)
8     << '\n';
9 }

```

Esse tipo de exemplo é útil quando se quer transformar uma regra matemática simples em uma malha sem precisar escrever cada coordenada manualmente.

observação

Mapeamento e malha

A função de mapeamento não é armazenada no objeto `Axis1D`. Ela é usada apenas para gerar as coordenadas. Depois que o eixo é construído, a malha fica representada pelas faces, centros e comprimentos calculados.

Como executar este exemplo

```
execução da malha por lei de potência  
cmake --build build --target run_man_powerLawGrid
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo separa bem duas etapas: primeiro se cria uma distribuição de faces por uma função; depois se entrega essa distribuição à FVGridMaker para montar a geometria do eixo.

seção 1.6

Reconstruindo centros com pesos alternados

Até aqui, mudamos principalmente a distribuição das faces. O exemplo `man_alternatingWeightedCenters1D` faz outra coisa: mantém uma distribuição simples de faces e altera a forma de reconstruir os centros dos volumes.

Esse exemplo não representa uma recomendação geral para resolver problemas físicos. Ele existe para mostrar que a `FVGridMaker` separa a distribuição das coordenadas primárias da regra usada para completar a geometria.

Leitura do programa

O programa começa com uma malha uniforme auxiliar, copia suas faces e depois cria um eixo novo usando uma regra de centros ponderados.

Faces de partida

As faces vêm de uma malha uniforme:

```
1 const Axis uniform_axis =  
2     fvgrid::uniform_axis_1d<Scalar>(nvol, xinit, xfinal);  
3  
4     std::vector<Scalar> faces;  
5     faces.reserve(uniform_axis.num_faces());  
6  
7     for (fvgrid::Size i = 0; i < uniform_axis.num_faces(); ++i) {  
8         faces.push_back(uniform_axis.face(i));  
9     }
```

faces uniformes de partida

A malha final terá as mesmas faces da malha uniforme. O que muda é a posição dos centros dentro de cada volume.

Regra de pesos alternados

A regra de pesos alternados é definida por uma função lambda:

```

1  const Scalar alpha = Scalar{0.20};
2
3  const auto alternating_weight = [alpha](fvgrid::Size p) {
4      if (p % static_cast<fvgrid::Size>(2) == 0) {
5          return alpha;
6      }
7
8      return Scalar{1.0} - alpha;
9  };

```

pesos alternados

Nos volumes pares, o centro é deslocado para uma fração α do volume. Nos volumes ímpares, usa-se $1 - \alpha$. Assim, os centros alternam de posição dentro dos volumes.

Construção do eixo

A malha é construída com `CentersFromFaces1D`:

```

1  const Axis axis = fvgrid::custom_axis_1d(
2      fvgrid::BasicCoordinates1D<Scalar>::faces(std::move(faces)),
3      fvgrid::CentersFromFaces1D{alternating_weight}
4  );

```

centros reconstruídos por pesos

As faces são as coordenadas primárias. O padrão `CentersFromFaces1D` usa a regra de pesos para calcular os centros.

Como interpretar a tabela

O programa imprime, para cada volume, as faces, o centro e o peso usado:

```

1  for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2      std::cout << std::right
3          << std::setw(id_width) << p
4          << std::setw(value_width) << axis.west_face(p)
5          << std::setw(value_width) << axis.center(p)
6          << std::setw(value_width) << axis.east_face(p)
7          << std::setw(value_width) << axis.cell_length(p)
8          << std::setw(value_width) << alternating_weight(p)
9          << '\n';
10 }

```

tabela com pesos

Como as faces vêm de uma malha uniforme, os comprimentos dos volumes continuam iguais. O que muda são as posições dos centros. Isso deixa claro que distribuição de faces e reconstrução de centros são decisões diferentes.

observação

Não confundir duas geometrias

O comprimento do volume depende das faces. A posição do centro depende da regra de

reconstrução. Neste exemplo, os comprimentos permanecem uniformes, mas os centros deixam de ficar nos pontos médios.

Como executar este exemplo

```
execução dos centros ponderados  
cmake --build build --target run_man_alternatingWeightedCenters1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo mostra uma parte mais interna da arquitetura 1D: a malha pode ter faces definidas de uma forma e centros reconstruídos por outra. Isso é útil para testar regras geométricas e deixar claro o papel de cada componente.

seção 1.7

Avaliando métricas simples de qualidade

Depois de construir várias malhas, convém olhar para algumas métricas simples de qualidade. O exemplo `man_quality1D.cc` compara três casos: malha uniforme, malha de Roberts e malha aleatória.

Essas métricas não decidem, sozinhas, se uma malha é adequada para um problema numérico. Elas servem como inspeção inicial. Uma malha pode passar nesses testes e ainda assim não ser boa para uma equação específica. Mesmo assim, os testes ajudam a detectar problemas grosseiros.

Leitura do programa

O programa calcula medidas globais e locais dos comprimentos dos volumes.

Malhas comparadas

O exemplo cria três eixos:

```
malhas comparadas
1  const Axis uniform_axis =
2      fvgrid::uniform_axis_1d<Scalar>(nvvol, xinit, xfinal);
3
4  const Axis roberts_axis = fvgrid::roberts_axis_1d(
5      fvgrid::NVol{nvvol},
6      fvgrid::Length{xfinal - xinit},
7      fvgrid::XInit{xinit},
8      fvgrid::Beta{beta},
9      fvgrid::VolumeCentered1D{}
10 );
11
12 const Axis random_axis =
13     fvgrid::random_axis_1d<Scalar>(nvvol, xinit, xfinal, seed);
```

A comparação é interessante porque os três casos têm o mesmo domínio e o mesmo número de volumes, mas distribuições diferentes.

Métricas calculadas

O programa calcula, entre outras quantidades:

- soma dos comprimentos dos volumes;
- erro de fechamento geométrico;
- menor comprimento de volume;
- maior comprimento de volume;

- razão entre maior e menor comprimento;
- maior razão local entre volumes vizinhos;
- desvio relativo dos comprimentos.

Essas métricas são calculadas a partir dos valores de `axis.cell_length(p)`.

```
varredura dos comprimentos

1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2   const Scalar dx = axis.cell_length(p);
3
4   dx_values[p] = dx;
5   metrics.sum_dx += dx;
6
7   if (dx < metrics.min_dx) {
8     metrics.min_dx = dx;
9     metrics.min_index = p;
10  }
11
12  if (dx > metrics.max_dx) {
13    metrics.max_dx = dx;
14    metrics.max_index = p;
15  }
16 }
```

A malha uniforme deve apresentar razão maior/menor próxima de 1. A malha de Roberts deve mostrar variação controlada. A malha aleatória pode apresentar saltos mais fortes.

Razão local

A razão local compara volumes vizinhos:

```
razão local

1 const Scalar local_ratio = (left > right)
2   ? left / right
3   : right / left;
```

Essa métrica é simples, mas útil. Uma malha pode ter razão global alta e ainda variar suavemente; também pode ter razão global moderada e apresentar um salto local forte. A razão local ajuda a enxergar esse segundo caso.

Erro de fechamento

O erro de fechamento é calculado como nos exemplos anteriores:

```
erro de fechamento

1 metrics.closure_error =
2   std::abs(metrics.sum_dx - metrics.expected_length);
```

Esse valor deve ser pequeno. Se a soma dos volumes não fecha o comprimento total do eixo, há um problema geométrico básico.

observação**Métrica não é veredito**

As métricas deste exemplo são alertas. Elas ajudam a encontrar irregularidades e erros de construção, mas não substituem testes numéricos do problema que será resolvido.

Como executar este exemplo

execução das métricas de qualidade

```
cmake --build build --target run_man_quality1D
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo é uma ponte entre geração de malhas e uso numérico. Antes de aplicar uma malha em uma equação, vale conferir se sua geometria tem o comportamento esperado.

seção 1.8

Exportando uma malha 1D para CSV

O último exemplo do capítulo mostra como exportar uma malha 1D para um arquivo CSV. Esse formato é simples, fácil de abrir em planilhas e conveniente para inspeção rápida com scripts externos.

O exemplo `man_axis1DCSV.cc` usa uma malha de Roberts e grava uma linha para cada volume. A intenção não é substituir formatos de visualização científica, mas oferecer uma saída textual simples para conferência.

Leitura do programa

O programa constrói uma malha, imprime um resumo no terminal, verifica a consistência geométrica e grava o arquivo CSV.

Nome e caminho do arquivo

O nome do arquivo é definido assim:

```
1 const std::filesystem::path csv_filename{"axis1d_roberts.csv"};
2 const std::filesystem::path csv_path =
3     std::filesystem::absolute(csv_filename);
```

A chamada `std::filesystem::absolute` é importante porque o programa imprime o caminho completo do arquivo. Assim, o usuário sabe exatamente onde o CSV foi gravado.

Quando o exemplo é executado pelo alvo do CMake, o arquivo é criado no diretório de execução do alvo, normalmente a pasta `build`.

Malha exportada

A malha escolhida é uma malha de Roberts:

```
1 const Axis axis = fvgrid::roberts_axis_1d(
2     fvgrid::NVol{nv},
3     fvgrid::Length{xfinal - xinit},
4     fvgrid::XInit{xinit},
5     fvgrid::Beta{beta},
6     fvgrid::VolumeCentered1D{}
7 );
```

Essa escolha deixa o CSV mais informativo do que uma malha uniforme, pois os comprimentos dos volumes variam ao longo do domínio.

Cabeçalho do CSV

O arquivo CSV contém apenas dados por volume:

```

cabeçalho do CSV
1 file << "volume,"
2   << "face_oeste,"
3   << "centro,"
4   << "face_leste,"
5   << "dx,"
6   << "distancia_centro_face_oeste,"
7   << "distancia_face_leste_centro\n";

```

O padrão da malha não é repetido em cada linha, porque ele é uma propriedade global do eixo. Ele aparece no resumo impresso no terminal, não no CSV.

Linhas do CSV

Cada linha representa um volume:

```

linhas do CSV
1 for (fvgrid::Size p = 0; p < axis.num_cells(); ++p) {
2   const Scalar west = axis.west_face(p);
3   const Scalar center = axis.center(p);
4   const Scalar east = axis.east_face(p);
5
6   file << p << ','
7     << west << ','
8     << center << ','
9     << east << ','
10    << axis.cell_length(p) << ','
11    << center - west << ','
12    << east - center
13    << '\n';
14 }

```

As colunas registram a face oeste, o centro, a face leste, o comprimento do volume e as duas distâncias entre centro e faces. Esses dados são suficientes para uma inspeção geométrica básica.

Resumo no terminal

Antes de gravar o arquivo, o programa imprime um resumo:

```

resumo com caminho completo
1 std::cout << "número de volumes : " << axis.num_cells() << '\n';
2 std::cout << "número de faces : " << axis.num_faces() << '\n';
3 std::cout << "xmin : " << axis.xmin() << '\n';
4 std::cout << "xmax : " << axis.xmax() << '\n';
5 std::cout << "comprimento : " << axis.length() << '\n';
6 std::cout << "padrão : " << axis.pattern_name() << '\n';
7 std::cout << "arquivo CSV : " << csv_path.string() << '\n';

```

Essa mensagem evita uma dúvida comum: o arquivo não é gravado na pasta `manual/`, mas no diretório de execução do programa.

observação**CSV como inspeção**

O CSV é útil para verificar a geometria fora do programa, mas não é o formato ideal para grandes malhas ou visualização científica. Para esses casos, formatos próprios de visualização, como VTK, são mais adequados.

Como executar este exemplo

```
execução da exportação CSV  
cmake --build build --target run_man_axis1DCSV
```

Após a execução, procure na saída do terminal a linha com o caminho completo do arquivo:

```
caminho impresso  
Caminho completo: /caminho/do/repositorio/build/axis1d_roberts.csv
```

Código completo do exemplo

O programa completo deste exemplo está disponível no repositório da FVGridMaker.



Este exemplo fecha o capítulo mostrando um fluxo prático: gerar uma malha, verificar sua consistência e exportar seus dados geométricos para inspeção externa.

Referências Bibliográficas

Referências Bibliográficas

- [1] Patankar, S. V. *Numerical heat transfer*. 1ª edição. Boca Raton, FL, USA: CRC Press, Inc., 1980 (ver [pág. 3](#))